

Ship the cheap thing that works: color thresholds over YOLOv8

How a two-day technical spike removed weeks of ML work from the critical path of an AI game coach MVP

Project	Role	When	Artifacts
AI decision-thinking coach for Honor of Kings	Product owner / solo builder	Aug 2026 • phase 0 → phase 1	PRD v1.1 • tech spec v1.1 • ticket AGE-45

TL;DR — The tech spec defaulted to training a YOLOv8 object detector to read the in-game minimap. Before committing to weeks of data labeling, I ran a validation spike on a real replay video. It showed the game already color-codes ally/enemy icons (blue/red rings), so a classic computer-vision approach — HSV color thresholding plus connected-component filtering — delivers the MVP requirement with **zero** training data. I re-scoped the spec (AGE-45): color thresholds for MVP, YOLOv8 deferred to v1.x as a precision upgrade. Result: milestone 0 passed and phase 1 unblocked, with the expensive ML path converted from a prerequisite into an option.

Context

Coach is a post-game, voice-first AI coach that teaches decision-making rather than mechanics. Its core loop — "what were you thinking when you died?" — depends on automatically extracting two things from a replay screen recording: **when the player died** and **where the enemies were just before**. Both signals live on the minimap, so minimap hero detection sits on the critical path of the entire product: no detection, no replay analysis, no coaching conversation.

The original tech spec made a reasonable-sounding default choice: train a YOLOv8 object detector to find hero icons. Reasonable — but expensive, and chosen before anyone had looked closely at real minimap pixels.

The decision

How should the MVP detect ally and enemy hero icons on the minimap? Two credible options:

	A. HSV color threshold	B. YOLOv8 training
Data cost	Zero — no labeled data needed	Hundreds–thousands of hand-labeled frames
Time to first result	Hours (tune thresholds on real frames)	Weeks (collect → label → train → iterate)
Compute	CPU, milliseconds per frame	GPU for training; heavier inference
Output	Ally/enemy positions only	Positions + hero identity + confidence
Key risk	Breaks if game changes color coding	Large upfront spend before any user value
Precondition	Icons must have stable color encoding — unverified at spec time	None, works on generic visuals

Option A is dramatically cheaper but hinges on one empirical question: does the game render enemies and allies with stable, machine-separable colors? That question is answerable in days, not weeks — which

made it the perfect target for a phase 0 spike.

Validation (phase 0, 2026-08-11)

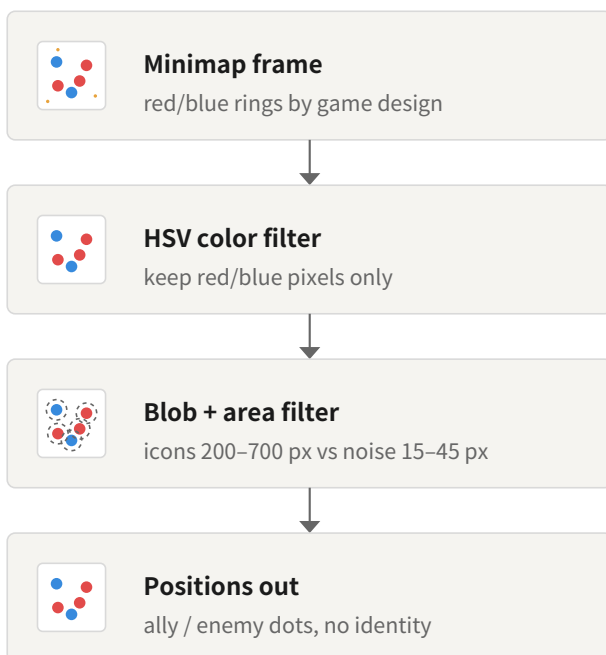
I ran the spike against a real 19.5-minute replay recording (1290×2796, 60 fps) using OpenCV on cropped minimap regions, sampled at three points across the match (~2, 10 and 15 minutes). Findings:

- 1. The color encoding is stable and by design.** Enemy heroes are wrapped in red rings, allies in blue rings, consistently across the whole match. HSV thresholding separates them reliably.
- 2. False positives are filterable by size.** Wards and gold-drop icons share similar hues, but hero icons occupy 200–700 connected pixels versus 15–45 for icon noise — a simple area filter removes them.
- 3. Free bonus signals.** Enemy icons disappear from the minimap exactly when they leave vision, so icon visibility doubles as the "enemy missing for N seconds" signal the spec had planned to model separately. And the game draws a red X at the death location — no position inference needed.

The two pipelines compared

Color thresholding

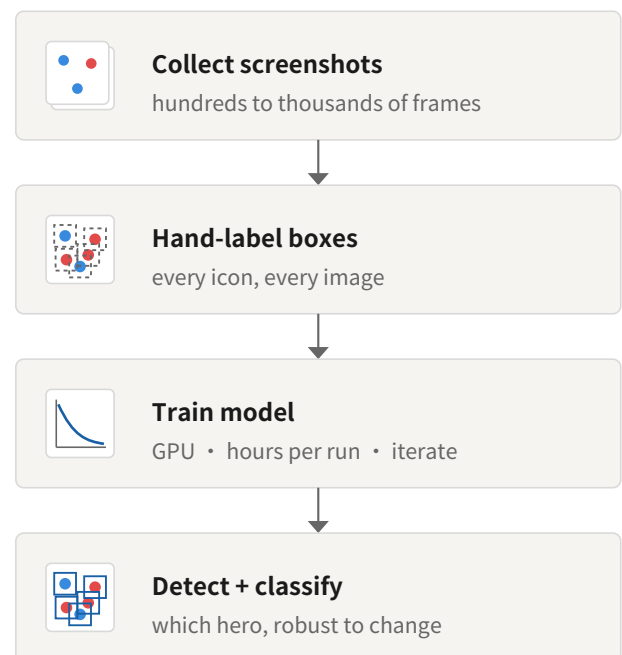
MVP • classic CV • zero labels



zero labels • CPU-fast • minutes to tune • no hero identity

YOLOv8 detection

v1.x • deep learning • labeled data



1000s of labels • GPU • weeks upfront • knows which hero

Same input, same MVP-level output. The right column's first three stages are pure upfront cost — they buy hero identity and robustness the MVP doesn't yet need.

Decision and rationale

I updated the tech spec (ticket AGE-45): **MVP uses HSV color thresholding + connected-component area filtering; YOLOv8 moves to the v1.x backlog as a precision optimization**, unblocked from the MVP timeline. Three principles drove this:

Validate before you invest. The spec's YOLOv8 default encoded an untested assumption — that minimap icons are hard to detect. A two-day spike was cheaper than discovering mid-labeling that the work was unnecessary.

Buy information, not infrastructure. The MVP question is "can we auto-extract death context at all?" — answerable with the crude method. Hero identity and robustness are v1.x questions; spending on them now would be paying for answers nobody has asked for yet.

Keep the expensive path open, not mandatory. Deferring is not deleting. YOLOv8 remains the plan of record for per-hero identification, and every frame processed by the MVP becomes future training data — the cheap path feeds the expensive one.

Tradeoffs and open risks

The choice consciously accepts limits, each with a monitoring or mitigation plan: the threshold approach yields **no hero identity** (enemy/ally only — sufficient for death-context analysis, revisit at v1.x); crop coordinates and thresholds are **calibrated to one device and UI version** (made configurable, not hardcoded, so a game UI update degrades gracefully); and **teammate disambiguation and continuous trajectory tracking remain unverified** (explicitly logged as open questions that do not block phase 1, to be validated on demand in phase 2).

Follow-up spike: hero identity without ML

The main limitation accepted above — no hero identity — turned out to be cheaper to lift than expected. Two observations reframed the problem: the match roster is known (10 heroes, visible in the loading screen and HUD), so identification is a pick-one-of-five problem, not a recognize-anyone problem; and each hero's minimap icon is a unique portrait, so the ring detector's output can be matched against a small self-captured template library.

I tested two matching methods on real icons from the same replay: crops at ~5:00 game time as templates, crops at ~10:00 as queries. **HSV histogram comparison** (compare color proportions, ignore arrangement) and **perceptual hash** (shrink to an 8×8 light/dark map, compare 64-bit structural fingerprints).



Matching matrices on real replay icons. Green = correct match; red = wrong or misleading. The ally warrior (bottom row) has no true match among the templates — the histogram wrongly flags 0.58 similarity to the fighter; the hash correctly finds no confident winner.

Findings. Both methods re-identified returning heroes across a 5-minute gap (mage: 0.95 correlation / 8-bit distance; fighter: 0.99 / 14). But the histogram showed its predicted failure mode on real data: it scored two different pale-haired heroes 0.68 similar, and handed an unknown hero a misleading 0.58 match — colors can lie, because different heroes dress in the same palette. The hash was fooled by neither, since it compares where features sit rather than what colors they use.

Resulting design rule: use the color check as a cheap "definitely not" filter, let the fingerprint make the actual call, and break remaining ties with priors — role positions in the opening minute (junglers in jungle), one-of-each-hero joint assignment, and tracking continuity between disappearances. This lifts the MVP's biggest accepted limitation using the same zero-training toolkit, shrinking what v1.x still needs YOLOv8 to buy.

Outcome

Milestone 0 achieved: a complete death event (time + pre-death enemy positions) was extracted from a real replay with zero manual marking, both pipelines independently validated. Phase 1 (core skeleton) started without any ML tooling, dataset, or GPU dependency. The same spike also killed a second over-engineered spec item — full-video scene-detection scanning, replaced by lightweight HUD-counter sampling with binary search after the original approach timed out twice on real footage. One validation exercise, two spec corrections, zero sunk cost.

What this shows about how I work: I treat a tech spec as a set of hypotheses, price the riskiest assumption, and buy the cheapest experiment that can change the plan — before the plan gets expensive.